System Setup & Functionality

1. Overall Setup

The system generates a weekly schedule for restaurant staff.

Time structure

- 7 days: Monday to Sunday
- 2 shifts per day:
  - Morning
  - Evening

Roles per shift

- Cashier
- Server

Employee attributes
Each employee is defined by:

- Skills (e.g., cashier, server)
- Availability (day + shift level)
- Maximum number of shifts per week
- Employment type (full-time / part-time)

Employment type is used as a proxy for experience and is incorporated into optimization.

2. Core Constraints (Hard Constraints)

The following constraints must always be satisfied:

- **Shift coverage**
  - Each shift must meet required staffing levels for each role
- **Skill matching**
  - Employees can only be assigned to roles they are qualified for
- **Availability**
  - Employees can only be assigned to shifts they are available for
- **One role per shift**
  - Each employee can hold at most one role in the same shift
- **Maximum workload**
  - Each employee cannot exceed their weekly shift limit

These constraints guarantee that all generated schedules are valid.

3. Optimization Objectives (Soft Constraints)

After satisfying all hard constraints, the system optimizes the schedule based on:

- **Stability**
  - Minimize unnecessary changes from the current schedule
- **Workload balance**
  - Distribute shifts more evenly across employees
- **User preferences**
  - Encourage or reduce specific assignment patterns when possible
- **Busy shift prioritization**
  - Prefer full-time employees for:
    - All evening shifts
    - All weekend shifts (Saturday & Sunday)

These are **soft constraints**:

- They guide the solution
- They can be overridden if needed to maintain feasibility

4. Flexible Input Handling

The system supports robust and user-friendly input parsing.

- **Case-insensitive**
  - ian, Ian, IAN → all valid
- **Alias support**
  - Days:
    - tue, tues, Tuesday → Tuesday
  - Shifts:
    - am, morning, day shift → morning
    - pm, night, evening → evening
- **Fuzzy matching**
  - Minor typos are automatically resolved
  - Example:
    - Iann → Ian

This ensures smooth interaction without strict input formatting.

5. Supported Update Types (User-Initiated Changes)

5.1 Shift Unavailability

Marks an employee unavailable for a specific day and shift.

Example

- Ian becomes unavailable for Sunday evening

Behavior

- The employee is removed from that shift
- Replacement staff are assigned if feasible

Explanation

- Ian is unavailable for Sunday evening, so Brian covers it instead.

5.2 Shift Direct Swap

Swaps assignments between two employees.

Example

- Brian (Monday evening) ↔ Ian (Tuesday evening)

Behavior

- Assignments are exchanged
- The system ensures:
    - Skill compatibility
    - Constraint satisfaction

Explanation

- Brian and Ian swapped Monday evening and Tuesday evening.

5.3 Back-to-Back Preference

Adds a preference related to consecutive shifts.

Definition

- Same-day: morning + evening
- Cross-day: evening → next morning

Behavior

- A preference is added
- The schedule adjusts if feasible
- The system can:
    - reduce back-to-back assignments
    - encourage back-to-back assignments

Penalty interpretation (-10 ~ 10)

- Positive → avoid
- Negative → prefer
- 0 → neutral (no preference)

Example (avoid)

- Avoid Eric working back-to-back shifts

Explanation

- Eric now has fewer back-to-back shifts
  (same-day: 2 → 1, cross-day: 1 → 0)

Example (prefer)

- Prefer Eric working back-to-back shifts

Explanation

- Eric now has more back-to-back shifts
  (same-day: 1 → 2, cross-day: 0 → 1)

5.4 Shift Preference (Flexible Granularity)

Supports both avoiding and preferring assignment patterns.

Supported scopes

1. **Shift type**
    - Example: prefer Alice → morning
    - Example: avoid Alice → morning
2. **Specific day**
    - Example: prefer Hannah → Sunday
    - Example: avoid Hannah → Sunday
3. **Specific day + shift**
    - Example: prefer Brian → Saturday evening
    - Example: avoid Brian → Saturday evening

Behavior

- The preference is stored
- The system tries to increase or reduce matching assignments

Penalty interpretation (-10 ~ 10)

- Positive → avoid
- Negative → prefer
- 0 → neutral (no preference)

Explanation examples

- Alice now works fewer morning shifts (3 → 1)
- Brian now works more Saturday evening shifts (0 → 1)

5.5 Hard Restriction (Pattern Unavailability)

A strict version of preference.

Example

- Ian cannot work evening shifts

Behavior

- Becomes a hard constraint
- All matching assignments become ineligible and will be removed if they exist.

Explanation

- Ian is no longer assigned to evening shifts (4 → 0)

6. Preference Accumulation

The system supports sequential updates.

Behavior

- Preferences are stored over time
- Future schedules consider all accumulated preferences

Example

- Step 1: avoid Alice morning
- Step 2: avoid Alice Sunday

Result

- Both constraints are applied simultaneously

7. Explanation Module

The system generates concise, natural explanations for each update.

Key characteristics

- Typically one sentences
- Focus on what changed
- Uses resolved names (after fuzzy matching)
- Includes before → after comparisons when relevant

What explanations include

- Availability changes
- Assignment replacements
- Preference impact
- Changes in assignment counts

What explanations do not include

- Long multi-step reasoning
- Full constraint breakdown
- All optimization details

8. Busy Shift Behavior

Full-time staff are preferred on:

- Evening shifts
- Weekend shifts

However:

- This is a soft constraint
- It may not always appear in explanations
- It is reflected in assignment outcomes when feasible

9. Key Design Principle

The system balances three priorities:

- **Feasibility**
  - All hard constraints must be satisfied
- **Preferences**
  - Soft constraints guide optimization
- **Stability**
  - Minimize unnecessary changes

Final schedules are always:

- Valid first (all hard constraints satisfied)
- Optimized second (soft objectives applied when feasible)

10. Multi-Week Continuity

The system supports scheduling across multiple weeks.

- The final schedule of the current week becomes the initial schedule for the next week (carried over directly before re-optimization)
- Preferences accumulate over time
- The system re-optimizes while preserving existing assignments when possible

This design ensures continuity and stability across weeks.